

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторне заняття №3
з навчальної дисципліни
"Технології розробки корпоративних Web-додатків"

Виконав:

студент 401-ТН

Стефанович Богдан Ігорович

Перевірив:

Тютюнник Петро Богданович

Полтава – 2026

Завдання

Мета роботи — реалізувати шар доступу до даних, бізнес-логіку та надати зовнішній інтерфейс для взаємодії з системою, використовуючи один із сучасних стандартів (REST, GraphQL або gRPC).

1. Реалізація шару доступу до даних (Data Access Layer)

- Налаштуйте підключення до бази даних через обрану ORM/ODM (Entity Framework, Hibernate, Mongoose, Drizzle тощо).
- Створіть моделі/сутності (Entities), що відповідають схемі БД з Лаб №2.
- Реалізуйте базові операції читання та запису (бажано через патерн Repository або ізольовані сервіси доступу до даних).

2. Реалізація бізнес-логіки (Service Layer)

- Створіть сервісний шар, який інкапсулює бізнес-правила.
- Вимога: API-контролери або резолвери не повинні містити складної логіки. Вони лише приймають дані, викликають сервіс і повертають результат клієнту.
- Застосуйте Dependency Injection (DI) для зв'язування компонентів.

3. Реалізація API (Оберіть один із варіантів) Виберіть підхід, який найкраще пасує до вашого тех стеку та вимог:

- Варіант А: RESTful API
 - Реалізуйте контролер для обраної сутності.
 - Забезпечте підтримку HTTP методів: GET (List/Single), POST (Create), PUT/PATCH (Update), DELETE.
 - Правильно використовуйте HTTP статус-коди (200, 201, 400, 404, 500).
 - Документація: Swagger (OpenAPI).
- Варіант Б: GraphQL API
 - Опишіть схему (Schema) з типами даних.
 - Реалізуйте Query для отримання даних (аналог GET).
 - Реалізуйте Mutation для зміни даних (аналог POST/PUT/DELETE).
 - Налаштуйте Resolvers для підключення схеми до бізнес-логіки.
 - Інтерфейс: GraphQL або Apollo Sandbox.
- Варіант В: gRPC API
 - Опишіть контракт сервісу у файлі .proto.

- Визначте повідомлення (Messages) для запитів та відповідей.
- Реалізуйте методи сервісу (Unary calls або Streaming, якщо необхідно).
- Інструментарій: підтримка генерації коду з proto-файлів.

4. Використання DTO (Data Transfer Objects)

- Незалежно від обраного протоколу, заборонено повертати клієнту "сирі" сутності бази даних.
- Використовуйте DTO (або message у gRPC, Type/Input у GraphQL) для передачі даних.
- Налаштуйте мапінг між сутностями БД та моделями API.

Очікуваний результат (Звіт та код):

- Посилання на Git-репозиторій.
- Скріншоти роботи API:
 - Для REST: Swagger UI або Postman.
 - Для GraphQL: Скріншот запиту (Query) та мутації (Mutation) у Playground.
 - Для gRPC: Скріншот виклику методу через gRPC клієнт (напр. Postman gRPC або BloomRPC).
- Приклади коду (фрагменти у звіті):
 - Визначення моделі API (DTO / Proto message / GraphQL Type).
 - Реалізація методу (Controller Action / Resolver / gRPC Service method).

Виконання

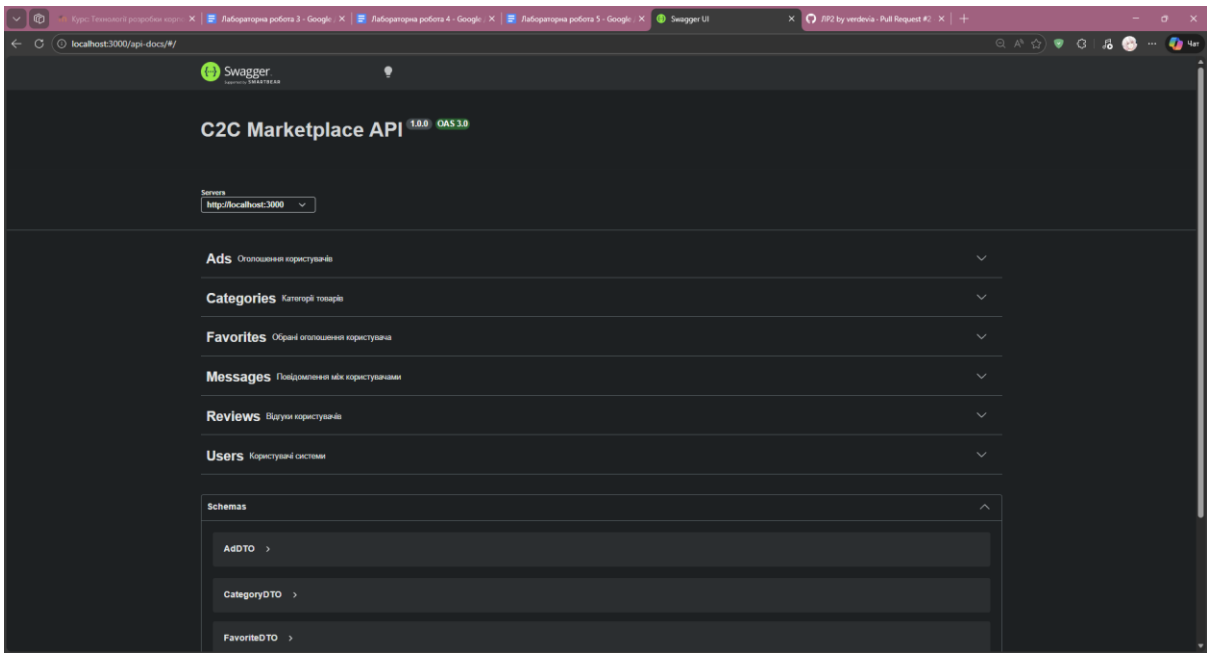


Рис. 1. – Swagger із усіма сутностями

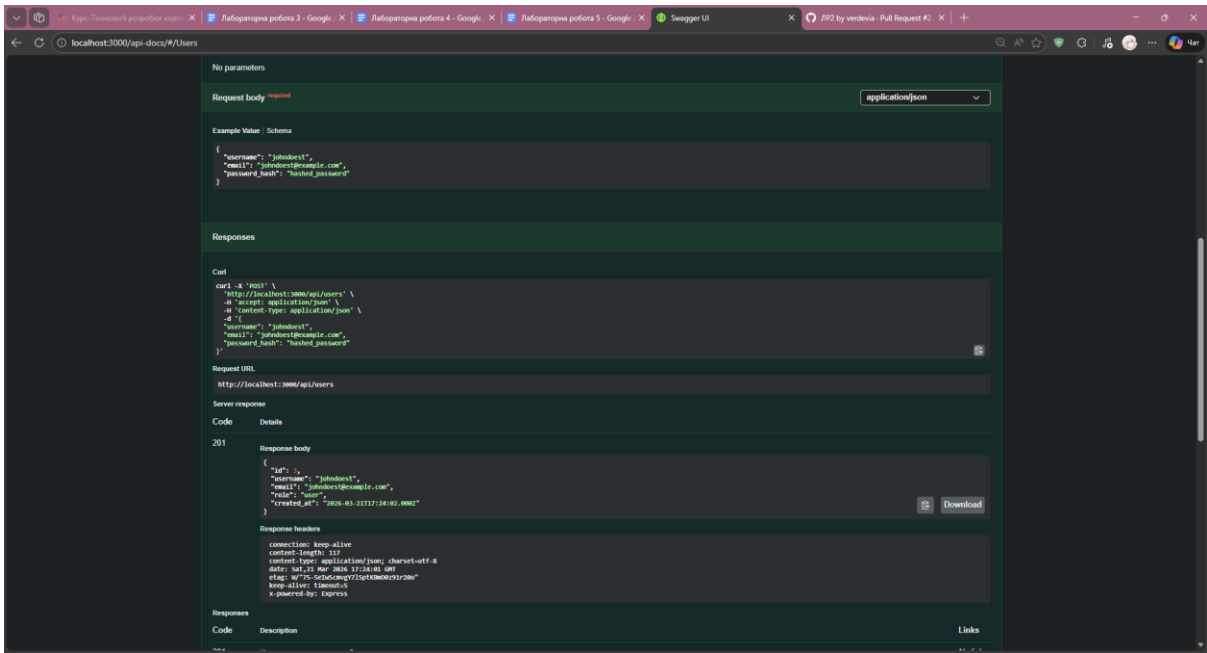
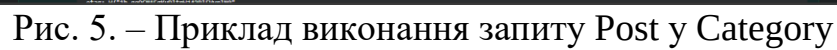


Рис. 2. – Приклад виконання запиту Post у User

3	johndoe1	johndoe1@example.com	hashed_password	user	2026-03-21 17:24:02
NULL	NULL	NULL	NULL	NULL	NULL

Рис. 3. – Результат виконання запиту у БД




```
.env X JS user.dto.js X
src > dto > JS user.dto.js > ...
1 class UserDTO {
2   constructor({ user_id, username, email, role, created_at }) {
3     this.id = user_id;
4     this.username = username;
5     this.email = email;
6     this.role = role;
7     this.created_at = created_at;
8   }
9 }
10
11 module.exports = UserDTO;
```

Рис. 8. – user.dto.js

```
JS ad.dto.js
JS category.dto.js
JS favorite.dto.js
JS message.dto.js
JS review.dto.js
JS user.dto.js
```

Рис. 9. – Аналогічні dto для кожної таблиці

```
VS Code: c2c-marketplace-api
File Edit Selection View Go Run Terminal Help
src > controllers > JS user.controller.js > ...
1 const express = require("express");
2 const router = express.Router();
3
4 const { userService } = require("../controllers");
5 const UserDTO = require("../dto/user.dto");
6
7 /**
8  * @swagger
9  * tags:
10  *   name: Users
11  *   description: Копіюється в cactemw
12  */
13
14 /**
15  * @swagger
16  * components:
17  *   schemas:
18  *     UserDTO:
19  *       type: object
20  *       properties:
21  *         id:
22  *           type: integer
23  *           example: 1
24  *         username:
25  *           type: string
26  *           example: johndoe
27  *         email:
28  *           type: string
29  *           example: johndoe@example.com
30  *         role:
31  *           type: string
32  *           example: user
33  *         created_at:
34  *           type: string
35  *           format: date-time
36  */
37
38 /**
39  * @swagger
40  * /api/users:
41  *   get:
42  *     summary: Отримати всіх користувачів
43  *     tags: [Users]
44  *     responses:
45  *       200:
46  *         description: Список користувачів
47  *         content:
48  *           application/json:
49  *             type: array
50  *             items:
51  *               $ref: '#/components/schemas/UserDTO'
```

Рис. 10. – user.controller.js із позначками для swagger

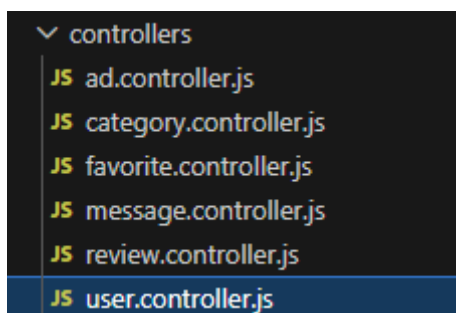


Рис. 11. – Аналогічні контролери для кожної таблиці

Посилання на репозиторій

[verdevia/TRVD_2026_401TN_Stefanovych_Labs_at_lab3](https://github.com/verdevia/TRVD_2026_401TN_Stefanovych_Labs_at_lab3)